

local intelligence documentation en

The documentation is organized by language. These pages include current implementation notes as well as historical proposals; a proposal is not proof that a feature or route exists. The MongoDB handover and project-start pages describe the current snapshot-persistence implementation.

Contents

Setup and implementation

- [Project start](#)
- [MongoDB persistence handover](#)
- [Prototype database connection and data model](#)
- [Database connection design examples \(historical\)](#)
- [Architecture and placement conventions](#)
- [Data lifecycle policy proposal](#)

Data model and API design

- [Local Intelligence data model v0.1](#)
- [Data model proposal v0.1](#)
- [Entity map v0.1](#)
- [Proposed API routes](#)
- [Model and API-route overview](#)
- [Event taxonomy and schema governance](#)

Behavior and simulation proposals

- [Tick-based simulation](#)
- [Intelligent signal routing](#)
- [Autonomy strategy](#)
- [Signal, rule, and autonomy example](#)
- [Example: Crows and Holy — interactive show activation](#)

Starting Local Intelligence

This guide describes how to run the prototype locally with MongoDB.

Prerequisites

- Node.js 20.19.0 or later and npm
- Docker with the dedicated MongoDB container `mon-mongo`, or your own MongoDB instance on port 27017
- Local ports 27017 and 3000 available

The existing `crowds-mongodb` container belongs to the Crowds system. This guide does **not** start, stop, or modify it. Local Intelligence uses `mon-mongo` and its own MongoDB database, `local_intelligence`.

Install and start the backend

From the repository root:

```
cd local-intelligence
npm install
```

The local `.env` file contains the connection string `mongodb://127.0.0.1:27017/local_intelligence`. For a fresh checkout, copy `.env.example` to `.env` and adjust the values if needed. `.env` contains local configuration and is excluded from Git.

Start only the dedicated MongoDB container:

```
docker start mon-mongo
```

If `mon-mongo` does not exist, you can instead start a new container with its own persistent volume:

```
docker run -d --name local-intelligence-mongo --restart unless-stopped -p 27017:27017 -v local-intelligence-mongo-data:/data/db mongo:7
```

Then, from the `local-intelligence` directory:

```
npm start
```

The server first waits for the database and then restores saved runs. On successful startup, the console displays MongoDB connected and Local Intelligence running on port 3000. If MongoDB is unreachable, the API server does not start. Stop the server with **Ctrl+C**; runs are saved again before shutdown.

Alternatively, start it from the repository root:

```
node local-intelligence/app.js
```

Example data

The supplied local .env sets SEED_DEMO=true. At startup, the server checks whether the example experiment run experiment-9x9-test exists. Only if it is missing is it created through the local API. Repeated starts therefore do not create the example data again. To run without example data, set SEED_DEMO=false. seedScript2.js is not run automatically.

Example API requests:

```
GET http://localhost:3000/experiment-runs/experiment-9x9-test/summary
GET http://localhost:3000/agents?experimentRunId=experiment-9x9-test
GET http://localhost:3000/neighborhoods?experimentRunId=experiment-9x9-test
```

The script1.html and script2.html interfaces in the repository root expect the API at http://localhost:3000.

Run the integration test with MongoDB reachable:

```
npm test
```

The test uses a separate database with the _test suffix and deletes its test document when finished.

Persistence scope and limits

The existing simulation logic processes an ExperimentRun synchronously as an object. To make this prototype database-backed with minimal risk, Local Intelligence saves each complete run as a snapshot in a MongoDB document in experimentruns, using the existing string ID. On startup, snapshots are loaded back into the runtime cache. API changes are saved before JSON responses and also periodically.

The current runtime does not yet persist agents, neighborhoods, signals, histories, propagation events, or metrics in normalized collections. The design supports **one backend server process per database**. Each run snapshot is limited to approximately 14 MiB of JSON; larger runs receive an error when saved. For production use or larger experiments, growing signals, histories, propagation events, and metrics should be moved into their own collections, and multi-process operation must be made safe. These are future architectural steps, not prerequisites for local handover operation.

Troubleshooting

- **MongoDB connection failed:** Check `docker ps`, port 27017 availability, and `MONGODB_URI` in `.env`. Do not use `crowds-mongodb` as a substitute.
- **Port 3000 is in use:** Stop the competing process or change `PORT` in `.env`.
- **Port 27017 is in use:** Use `docker ps` to see whether another MongoDB container is using the host port. Do not modify the Crowds container; configure a separate available MongoDB port and update `MONGODB_URI`.
- **Dependencies are missing:** Run `npm install` in the `local-intelligence` directory.
- **Seeding failed:** Check the server console and confirm that port 3000 is available. The example run is created only if it does not already exist.

For details about the current persistence design, see [MongoDB Persistence Handover](#) and the [repository documentation](#).

MongoDB Persistence Handover

Implemented scope

- At startup, Mongoose connects to MongoDB through `MONGODB_URI`. If MongoDB is unavailable, the HTTP server does not start.
- Existing experiment runs are loaded from the `experimentruns` collection into the simulator's existing in-memory cache.
- Before JSON API responses are sent, affected experiment runs are saved as snapshots. A periodic flush saves all runs currently held in the process-local cache, and runs are saved again on a normal `SIGINT` or `SIGTERM` shutdown.
- Each snapshot uses the experiment run's existing string ID as MongoDB `_id`, preserving API and seed IDs.

- The demo seed is optional and configurable; it checks whether the fixed example run already exists before creating it.
- `.env` is local and excluded from Git. `.env.example` documents the configuration without secrets.

Deliberate handover decision

The simulation logic was not rewritten in this finalization pass to perform asynchronous updates across multiple collections. Instead, each run's complete runtime object is stored in the `data` field of a MongoDB document in `experimentRuns`. This preserves the existing synchronous controllers, rules, and simulation ticks, and restores them from MongoDB after a restart.

This is MongoDB-backed persistence for the current prototype, **not a normalized production architecture**. The current runtime does not yet persist agents, neighborhoods, signals, or events in normalized collections. MongoDB limits a single BSON document to 16 MiB, so the implementation applies a precautionary cap of approximately 14 MiB of JSON per run snapshot. The design supports one backend server process per database. Because snapshots are periodically flushed, an abrupt process or host failure may lose the most recent few seconds of changes.

Outside the current handover scope

- Multiple API instances or a coordinated shared cache
- Large runs with unbounded signal, history, or event arrays
- Separately persisted agent, neighborhood, signal, and event documents
- Distributed transactions, archiving, and lifecycle retention
- Session, media, simulator, recovery, and analytics routes described in design documents

Recommendations for the next technical handover

1. As data grows, move histories and propagation/protocol events into separate collections and make data access asynchronous.
2. Introduce snapshot-schema version migrations before making incompatible changes to the data format; `snapshotVersion` is reserved for this purpose.
3. Before running multiple backend instances, remove the in-memory cache or replace it with an explicit consistency and locking model.
4. Align lifecycle cleanup with the actual snapshot data and test it, including dependent data.

5. Before production use, add authentication, access controls, backups, monitoring, and appropriate MongoDB user permissions.

Local operation and verification

See [Project Start](#) for local setup instructions. With MongoDB running, `npm test` in `local-intelligence` runs an integration test against a separate database with the `_test` suffix.

Prototype database connection and data model

Source note: The source file named `prototypDatabaseConnectionExplain.md` contains a data-model overview rather than an explanation of the database connection. This document corrects that mismatch and separates verified prototype behavior from conceptual or proposed entities.

1. Current database connection and persistence

The prototype uses Mongoose to connect to MongoDB. `database.js` reads `MONGODB_URI` from the environment and fails with a configuration error if it is missing. The optional `MONGODB_SERVER_SELECTION_TIMEOUT_MS` setting defaults to 10000 milliseconds.

At startup, `app.js` loads `local-intelligence/.env`, connects to MongoDB, restores experiment-run snapshots, and then starts the HTTP server. The in-memory `store.experimentRuns` array is the working store for runs. The persistence service writes each run as a data snapshot in the `experimentruns` collection and also copies selected fields into the document for querying. API responses associated with a run trigger persistence, and a periodic flush runs every 3000 ms by default (configurable with `PERSISTENCE_FLUSH_INTERVAL_MS`). Graceful shutdown saves all runs and disconnects from MongoDB.

This is snapshot-based persistence for experiment runs. The existence of other Mongoose model schemas does not mean that all runtime data is currently written to separate collections through those models.

2. Data-model layers

The source sketch groups the model concepts as follows:

1. **Experiment:** `ExperimentRun`, `ExperimentParticipant`.

2. **Agents and neighborhoods:** Agent, AgentState, AgentStateHistory, Neighborhood, NeighborhoodRelation.
3. **Signals and propagation:** Signal, PropagationEvent, Rule, RuleSet, Threshold.
4. **Observations and results:** Observation, ObservationMetric, CollectiveBehaviorResult.
5. **Operations and diagnostics:** ProtocolEvent, TechnicalWarning, FailureState.
6. **Supporting or proposed concepts:** State, Session, Media.

The implementation names the relationship model `NeighborhoodRelation`; the source sometimes calls it `NeighborRelation`.

3. Central conceptual relationship

```
ExperimentRun
├─ ExperimentParticipant
├─ Agent — AgentState / AgentStateHistory
│   └─ Neighborhood — NeighborhoodRelation — Agent
│       └─ Signal — PropagationEvent
├─ Observation — ObservationMetric
├─ CollectiveBehaviorResult
├─ ProtocolEvent
├─ TechnicalWarning
└─ FailureState
```

This is a domain-level diagram, not a guarantee that every line is a persisted database reference or a one-to-many constraint.

4. ExperimentRun and participants

`ExperimentRun` represents one experiment execution. Its schema includes run status (created, running, paused, finished, failed, or cancelled), lifecycle information, environment (test, rehearsal, research, or production), `activeRuleSetId`, statistics, diagnostics, and an optional `sessionId`. That `sessionId` is an external session identifier; it is not a reference to a local `Session` model. At runtime, the complete run is stored as a snapshot in the MongoDB collection `experimentruns`.

`ExperimentParticipant` is a separate model definition, but the snapshot persistence path does not write it separately.

5. Agents, states, and neighborhoods

Agent represents a device or participant in the simulation. Its schema includes `experimentRunId`, `deviceId`, `stateId`, `neighborhoodId`, `position`, autonomy settings, status (offline, online, or busy), and `lastSeen`. An agent's state is represented by `AgentState`; `AgentStateHistory` is intended to record changes over time. The agent schema has one `neighborhoodId` field, but the broader domain rule and relationship behavior should still be checked against the relevant workflows.

Neighborhood represents a local interaction context. `NeighborhoodRelation` represents a direct connection between agents. The source describes an agent as belonging to one neighborhood in the proposed architecture; treat that as a domain rule to validate against the relevant workflow, rather than a universal database constraint established by this diagram.

Example local position:

```
{
  "position": {
    "row": 2,
    "col": 3
  }
}
```

6. Signals, propagation, and rules

Signal represents a message associated with a run and potentially with source and target agents. The source sketch names `sourceAgentId`, `targetAgentId`, `experimentRunId`, `neighborhoodId`, and `correlationId` as relevant relationships. `PropagationEvent` records a propagation or processing step and can refer to the signal, participating agents, triggered rules, status, and correlation identifier. Treat this list as a domain description; verify individual fields against each schema before relying on them.

Rule represents one behavioral rule, `RuleSet` groups rules, and `Threshold` can hold an activation boundary. `ExperimentRun.activeRuleSetId` identifies the active rule-set reference. Whether experiment-level control messages and agent-to-agent messages should share the same `Signal` model remains an open design question.

7. Observations, results, and diagnostics

Observation captures an observed event or situation; `ObservationMetric` stores associated measurements. `CollectiveBehaviorResult` holds an analysis outcome, such as synchronization, propagation, clustering, or consensus.

`ProtocolEvent` records significant domain or technical events. The source notes list names such as `SESSION_CREATED`, `SESSION_CONNECTED`, `SESSION_DISCONNECTED`, `SIGNAL_CREATED`, `SIGNAL_STATUS_CHANGED`, `PROPAGATION_CREATED`, `PROPAGATION_STATUS_CHANGED`, `NEIGHBORHOOD_CREATED`, `NEIGHBORHOOD_UPDATED`, `RULESET_CREATED`, `RULESET_ASSIGNED`, and `AGENT_REPLACED` as examples. These names are not all confirmed by the current route implementation; `AGENT_REPLACED`, for example, appears in the neighborhood service. `TechnicalWarning` represents a technical warning, while `FailureState` represents a failure condition. These concepts have different purposes and should not be treated as interchangeable.

Concept	Meaning
<code>ProtocolEvent</code>	What happened in the system?
<code>TechnicalWarning</code>	What technical warning or symptom was detected?
<code>FailureState</code>	What confirmed failure condition is present?

For example, an agent heartbeat timeout may generate a `TechnicalWarning`; a confirmed outage may be represented by a `FailureState`; and a replacement operation may be recorded as a `ProtocolEvent`. The exact events emitted depend on the implemented workflow.

8. Implemented models versus proposals

The current [model index](#) exports schemas for `Agent`, `AgentState`, `AgentStateHistory`, `CollectiveBehaviorResult`, `ExperimentParticipant`, `ExperimentRun`, `FailureState`, `Neighborhood`, `NeighborhoodRelation`, `Observation`, `ObservationMetric`, `PropagationEvent`, `ProtocolEvent`, `Rule`, `RuleSet`, `Signal`, `State`, `TechnicalWarning`, and `Threshold`. This confirms that those model definitions exist; it does not establish that every proposed association or workflow is complete.

The source also lists `Session` and `Media`, but neither appears as a dedicated exported model in the current model index. The source's proposed `Session` fields include `type`, `socketId`, `performerId`, `isConnected`, `lastSeen`, and `disconnectReason`, with a lifecycle of `created`, `connected`, `disconnected`, and `reconnected`. These are design notes, not verified fields or lifecycle records in a local `Session` model. Consider `Session` and `Media` proposals or external concepts unless and until local model definitions and workflows are added. Likewise, replacement-agent relation transfer and other lifecycle behaviors should be treated as design statements unless verified in their implementation.

9. Example end-to-end domain flow

```
ExperimentRun
  ↓
Neighborhood
  ↓
Agent A
  ↓ creates
Signal
  ↓ propagates
PropagationEvent
  ├── Rule / RuleSet
  └── Agent B
      ↓
Observation
  ↓
ObservationMetric
  ↓
CollectiveBehaviorResult
```

Important system events may also be recorded as `ProtocolEvent` entries. A technical problem may produce a `TechnicalWarning` and, if a failure is confirmed, a `FailureState`.

For a concise entity overview, see [Entity map](#). For a concrete, explicitly illustrative signal/autonomy scenario, see [Signal, autonomy, and follow-up example](#).

Database connections

Status: The examples below are earlier design sketches for a normalized MongoDB implementation; they are **not** the current API implementation. Local Intelligence currently persists complete `ExperimentRun` snapshots. See the [current architecture overview](#) and [run-persistence adapter](#) for the implemented persistence path. In the current API, `row` and `col` must be supplied when creating an Agent. The sample below is historical and must not be treated as a current endpoint contract.

Agents

Register an Agent

```
/* REGISTER AGENT */
router.post("/agents", async (req, res) => {
  try {
    const agent = await Agent.create({
```

```

    deviceId: req.body.deviceId,

    // Initial state
    stateId: null,

    // Agent is not assigned to a neighborhood yet
    neighborhoodId: null,

    // Position will be assigned later
    position: {
      row: req.body.row ?? null,
      col: req.body.col ?? null,
    },

    // Device information
    deviceInfo: {
      model: req.body.deviceInfo?.model ?? null,
      platform: req.body.deviceInfo?.platform ?? null,
      version: req.body.deviceInfo?.version ?? null,
    },

    direction: req.body.direction ?? 0,

    status: "online",

    lastSeen: new Date(),

    metadata: req.body.metadata ?? {},
  });

  res.status(201).json(agent);

} catch (error) {
  // Duplicate deviceId
  if (error.code === 11000) {
    return res.status(409).json({
      error: "Agent with this deviceId already exists",
    });
  }

  res.status(500).json({
    error: "Failed to register agent",
    details: error.message,
  });
}
});

```

Update an Agent's state

```

router.patch("/agents/:id/state", async (req, res) => {
  try {
    const agent = await Agent.findById(req.params.id);

```

```

if (!agent) {
  return res.status(404).json({
    error: "Agent not found",
  });
}

const previousState = agent.stateId;

agent.stateId = req.body.stateId;
agent.lastSeen = new Date();

await agent.save();

res.json(agent);

} catch (error) {
  if (error.name === "CastError") {
    return res.status(400).json({
      error: "Invalid agent ID",
    });
  }

  res.status(500).json({
    error: "Failed to update agent state",
    details: error.message,
  });
}
});

```

The code examples intentionally retain their original normalized-model design. They are reference sketches, not evidence that these endpoints or per-Agent MongoDB documents are implemented in the current Local Intelligence API. For lifecycle policy and its implementation status, see the [data lifecycle policy](#).

Architecture and placement conventions

This document describes the current architecture of the Local Intelligence prototype and where new files belong. It combines the current MongoDB snapshot-persistence design with the existing architectural conventions.

Core rule

`routes/` and `controllers/` are HTTP-facing layers. Business logic belongs in `domain/` or `services/`. The `middlewares/` directory is reserved for genuine Express middleware, typically functions with a `(req, res, next)` signature.

Current structure

<code>local-intelligence/</code>	
<code> routes/</code>	Express routes and handler mappings
<code> controllers/</code>	HTTP handlers and request/response adapters
<code> repositories/</code>	In-memory run access and domain lookup helpers
<code> constants/</code>	Domain state, action, propagation, and status values
<code> domain/</code>	
<code>rules/</code>	Rule evaluation and execution
<code>signals/</code>	Signal processing and propagation
<code>behavior/</code>	Agent and swarm behavior without HTTP dependencies
<code> services/</code>	
<code>autonomy/</code>	Per-agent autonomy decisions and actions
<code>simulation/</code>	Simulation ticks and repeated simulation runs
<code>analysis/</code>	Agent, signal, neighborhood, and collective analysis
<code>logging/</code>	State, propagation, warning, and failure event logging
<code>scheduling/</code>	State-triggered and delayed actions
<code> utils/</code>	Small technical helpers without business workflows
<code> models/</code>	Mongoose model definitions
<code> config/</code>	Database connection configuration

Responsibilities

`routes/`

Routes define the HTTP method, path, and handler mapping. They should not look up the store, mutate domain state, or execute simulation logic.

```
router.post("/rules", createRule);
```

`controllers/`

Controllers read request data, call repositories, domain functions, or services, and build HTTP responses. The current simulator mutates a process-local experiment-run object. Persistence middleware saves the affected run snapshot to MongoDB before sending a JSON response.

repositories/

Repositories encapsulate lookups and mutations within the existing in-memory run structures. They use the restored `store.js` cache; MongoDB persistence is handled separately by `services/runPersistence.js`:

```
MongoDB experiment-run snapshots
-> restore on startup
    -> store.js process-local cache
        -> controllers / domain / services
            -> snapshot write-through on JSON responses
            -> periodic snapshot flush
```

Current repository modules include:

```
repositories/
  experimentRunRepository.js
  agentRepository.js
  neighborhoodRepository.js
  rulesetRepository.js
  ruleRepository.js
  signalRepository.js
```

The persistence adapter stores one serialized experiment run per MongoDB document, using the existing string run ID. This is a prototype handover design, not normalized persistence of separate Agent, Neighborhood, Signal, or event documents. The cache is process-local; the current setup is intended for one backend process per database. Snapshot data is subject to a 14 MiB JSON-size safety limit.

Repositories contain data access only. Rule decisions, signal propagation, and simulation belong in `domain/` or `services/`. See [the repository and persistence notes](#) and [the run-persistence adapter](#).

domain/

This is the business core of the local intelligence system:

- `domain/rules/`: determines which rules apply and which actions they trigger.
- `domain/signals/`: processes signals and determines propagation behavior and scope.
- `domain/behavior/`: implements agent and swarm behavior without HTTP concerns.

The domain may use technical services such as logging, but it must not import Express routes.

services/

Services coordinate larger workflows:

- `autonomy/`: per-agent decisions and autonomous actions.
- `simulation/`: individual ticks and repeated simulation runs.
- `analysis/`: measurement and interpretation of agent, signal, neighborhood, and state distributions.
- `logging/`: consistent recording of runtime events on an experiment run.
- `scheduling/`: scheduling and processing state-triggered actions.

utils/

Only small, preferably stateless helper functions belong here, for example geometric calculations in `utils/geometry.js`.

constants/

Reusable domain values belong here so code does not depend on hard-to-read magic numbers or repeated status strings:

```
constants/  
  actions.js  
  statuses.js  
  propagation.js
```

State IDs are still resolved through `stateHelpers.js` using definitions in `store.js`. The existing store structure remains unchanged.

Important dependencies

The main simulation and signal-processing flow is:

```
simulation tick  
  -> autonomy decisions and signal analysis  
    -> signal creation / signal engine  
      -> neighborhood analysis  
      -> rule evaluation and execution  
        -> behavior and state changes  
        -> propagation  
      -> event logging
```

`services/analysis/analyzeNeighborhood.js` is used by both autonomy and the signal engine, so it belongs in `analysis/`.

Signal-processing flow

A signal generally follows this sequence:

```
Signal creation
-> processSignal
-> neighborhood analysis
-> rule evaluation
-> rule execution
-> state change
-> optional propagation
-> logging
```

Specifically:

1. `createSignal.js` validates the source and target, creates the signal, and adds it to the current run.
2. `signalEngine.js` sets the signal status to processing and begins processing it.
3. `analyzeNeighborhood.js` calculates local agent and state information.
4. `evaluateRules.js` checks the active RuleSet, scope, threshold, and neighborhood conditions.
5. `executeRule.js` runs a matching action, such as activation, blocking, synchronization, or propagation.
6. Affected agents and the signal receive updated state or status values.
7. During propagation, new signals are created with a decremented TTL and an incremented hop count.
8. `runLogger.js` records state, propagation, warning, and failure events on the run.

State triggers and delayed signals

Rules can react to state changes. The condition is stored under `trigger`, the action under `action`, and a signal to send later under `appendedSignal`:

```
{
  enabled: true,
  scope: "agent",
  agentId: "agent-a",
  trigger: {
    type: "state_changed",
    fromState: "inactive",
    toState: "active",
  },
  action: "send_signal",
  appendedSignal: {
    delayMs: 5000,
    targetAgentId: "agent-x",
```

```
    signalType: "follow_up",
    signalPayload: {
      strength: 1,
    },
    signalProperties: {
      propagationMode: "unicast",
      propagationScope: "neighborhood",
    },
  },
}
```

The flow is:

```
State change
-> evaluate the state_changed trigger
-> store appendedSignal as pending
-> wait until a simulation tick
-> call createSignal when executeAt is due
```

This does not use an uncontrolled `setTimeout`. Pending actions are stored in `run.scheduledActions` and processed by `simulationTick.js`. Therefore, `delayMs` measures real elapsed time, but execution occurs on a simulation tick after the action is due.

Import rules

- Routes import controllers.
- Controllers import repositories, domain functions, and services.
- Direct store imports belong in repositories or in the explicitly designated store layer.
- Domain code may import services and utilities, but never routes.
- Services may import domain code, other services, and utilities.
- Avoid introducing circular dependencies.
- Calculate relative imports from the importing file's location.
- After moving files, check every static and dynamic import.

Where should a new file go?

- Does it process req, res, or next? Use `routes/` or `controllers/`.
- Does it make a business decision about rules, signals, or behavior? Use `domain/`.
- Does it coordinate multiple business steps or simulation ticks? Use `services/`.

- Is it a small, stateless calculation? Use `utils/`.
- Is it cross-cutting Express infrastructure such as authentication or error handling? Use `middlewares/`.

Naming conventions

Use descriptive camelCase filenames such as `signalEngine.js`, `runLogger.js`, and `geometry.js`. Avoid abbreviations and unclear names.

Validation after structural changes

From the repository root, run at least these checks:

```
find local-intelligence -name '*.js' -print0 | xargs -0 -n1 node --check
node -e "import('./local-intelligence/app.js')"
```

Only run a startup check when the server process will be intentionally stopped or managed separately afterward. For the current database-backed application, importing `app.js` is distinct from starting the server and connecting to MongoDB.

Related documents: [data lifecycle policy](#) and [database connection design examples](#).

Data lifecycle policy v0.1

1. Purpose and implementation status

This document defines a proposed retention and lifecycle strategy for Crowds operational, experiment, research, rehearsal, test, and production data. Its purpose is to retain data only as long as necessary while preserving information with research or operational value.

The policy describes intended behavior; it is **not automatically enforced for Local Intelligence's current MongoDB snapshot store**. The current application persists complete experiment-run snapshots in the `experimentruns` collection. Lifecycle cleanup helpers exist, but they operate on separate Mongoose collections and are not wired to snapshot storage or exposed by the current API. The proposed Session TTL is also not active. See [the current run-persistence adapter](#), [the lifecycle policy configuration](#), and [the architecture overview](#).

The policy covers:

- Protocol events
- Session data
- Device and agent data
- Experiment runs
- Propagation events
- Signals
- Observations
- Analytics summaries
- Simulator outputs
- Test data
- Rehearsal data
- Production operational data

2. Experiment environments

Each experiment run should identify its environment. The supported values are:

- test
- rehearsal
- research
- production

The environment determines the intended retention and cleanup policy.

3. Lifecycle policy

Environment	Intended retention	Raw data	Summary	Intended action
Test	7 days	Delete	Not required	Delete the experiment and its dependent data
Rehearsal	30 days	Delete	Keep important summaries	Delete raw data; retain summaries

Environment	Intended retention	Raw data	Summary	Intended action
Research	Long-term; policy configuration currently specifies 3,650 days	Archive	Retain long-term	Archive for analysis
Production	Operational policy	Retain as required	Retain	Retain according to operational requirements

These are policy targets, not a statement that Local Intelligence currently applies those periods to its snapshots.

4. Test data

Test data is temporary. The policy allows completed, failed, or cancelled test experiments to be deleted after seven days. Dependent raw data includes:

- ExperimentParticipant
- Signal
- PropagationEvent
- Observation
- ProtocolEvent
- TechnicalWarning
- FailureState

The associated ExperimentRun is deleted after its dependent records. Running experiments must never be deleted automatically. The test cleanup process must not delete research or production experiments.

5. Rehearsal data

Rehearsal data is generated during preparation and system validation before real operational events. Raw rehearsal data may be deleted after 30 days. Important experiment summaries, results, and metrics should be retained for operational review and future comparison.

6. Research experiment data

Research experiments have long-term value. Retain:

- Experiment metadata
- Experiment summaries and results
- Experiment statistics
- Important analytics summaries
- Relevant protocol events
- Relevant propagation data

When possible, archive raw research data rather than permanently deleting it. The current policy configuration sets a nominal retention period of 3,650 days and an archive action; that configuration is not, by itself, an active archival job.

7. Production operational data

Production data is retained according to operational and commercial requirements. The test cleanup process must not delete production data. Minimize device information to fields required for system operation, experiment correlation, and diagnostics.

8. Session data

The policy proposes that disconnected sessions be treated as temporary operational data and removed after seven days, while active sessions remain. **This Session TTL is not implemented in the current Local Intelligence API:** there is no local Session model or TTL index. An optional external `sessionId` field on an experiment run does not mean that local session records or TTL cleanup exist.

9. Protocol events

Classify protocol events according to their purpose. Runtime events may need shorter retention. Retain or archive events associated with research or production experiments when they are needed for:

- Experiment reconstruction
- Debugging
- Analytics
- Auditing
- Research

10. Propagation events

Propagation events represent signal propagation between agents and are treated as raw operational or experiment data. Their intended lifecycle depends on the experiment environment:

- Test: delete after the test retention period
- Rehearsal: delete after the rehearsal retention period
- Research: archive for long-term analysis
- Production: retain according to operational requirements

11. Device data minimization

Minimize agent and device data. Store only information required for:

- Agent identification
- Communication
- Position tracking
- Direction handling
- Experiment correlation
- Operational diagnostics

Do not collect unnecessary device metadata.

12. Cleanup procedures and current limitations

The policy describes these endpoint shapes:

Manual test cleanup (proposed)

```
DELETE /experiments/:id/test-run
```

This proposed endpoint would delete a completed test experiment and its dependent raw data.

Automatic test cleanup (proposed)

```
POST /lifecycle/cleanup/test-experiments
```

This proposed process would remove test experiments older than the configured retention period, limited to:

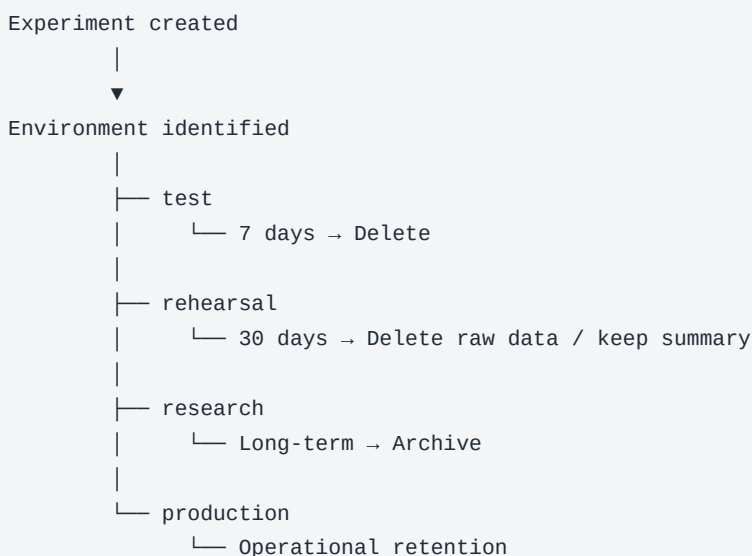
```
environment = test
status = finished | failed | cancelled
```

Running experiments are excluded. These lifecycle routes are not part of the current API. Repository cleanup helpers query normalized Mongoose models and collections; they do not clean the complete-run snapshots used by the current application. In addition, the manual helper only rejects running runs rather than enforcing the full terminal-status list above. Do not rely on these helpers as enforcement of the policy for current Local Intelligence data.

13. Data safety rules

1. Test cleanup must never delete research experiments.
2. Test cleanup must never delete production experiments.
3. Running experiments must never be deleted automatically.
4. Research summaries should be retained long-term.
5. Important rehearsal summaries should remain available after raw-data cleanup.
6. Minimize device data.
7. Archive data with long-term research value instead of deleting it where possible.
8. Log cleanup operations in production environments.

14. Lifecycle flow



15. Current implementation

The repository contains these policy and service building blocks:

- `ExperimentRun.environment` identifies the data environment in the Mongoose model and the persisted snapshot metadata.
- `dataLifecyclePolicy.js` defines retention-policy values, including 3,650 days for research.
- `dataLifecycleService.js` contains a manual cleanup helper for normalized Mongoose collections.
- `cleanupTestExperiments.js` contains an automatic-cleanup helper for expired test experiments.
- A MongoDB TTL policy for disconnected Session records is only a proposal; there is no local Session model or active TTL index in the current Local Intelligence API.

Local Intelligence implementation status: The application persists complete experiment-run snapshots in `experimentruns`. The cleanup helpers above are not connected to that snapshot storage, and the proposed lifecycle routes are not mounted in the current API. The policy must not be treated as automatically enforced for Local Intelligence data. The [run-persistence adapter](#) shows the current persistence path.

The intended design separates lifecycle policy from individual business routes so future retention rules can be extended without changing every data model.

16. Future improvements

Future versions should consider:

- Scheduled cleanup jobs
- Rehearsal-specific cleanup
- Data archiving
- Export before deletion
- Data anonymization
- Cleanup audit logs
- Configurable retention periods
- Production data-retention policies
- Storage for archived research datasets
- Lifecycle monitoring and metrics

17. Summary

The proposed Crowds data-lifecycle strategy distinguishes temporary operational data from long-term research and production data. Test data is intended for deletion after a defined retention period. Rehearsal raw data may be deleted while important summaries are retained. Research data should be archived for long-term use. Production data is retained according to operational requirements.

The strategy uses `ExperimentRun.environment` to associate a consistent policy with each run. For Local Intelligence, this remains a policy and set of cleanup building blocks, not active retention enforcement over MongoDB run snapshots.

Related documents: [architecture and placement conventions](#) and [database connection design examples](#).

Local Intelligence Data Model v0.1

Status: Descriptive data-model overview (version 0.1). This document describes the model concepts and relationships as presented in the source; the overview is not a guarantee that every described relationship is implemented exactly as shown.

1. Overall Structure

The data model can be divided into the following layers:

1. Experiment layer

- `ExperimentRun`
- `ExperimentParticipant`

2. Agent and Neighborhood layer

- `Agent`
- `AgentState`
- `AgentStateHistory`
- `Neighborhood`
- `NeighborRelation`

3. Signal and propagation layer

- `Signal`
- `PropagationEvent`

- Rule
- RuleSet
- Threshold

4. Observation and results layer

- Observation
- ObservationMetric
- CollectiveBehaviorResult

5. Operations, failure, and logging layer

- ProtocolEvent
- TechnicalWarning
- FailureState

6. Session and media layer

- Session
- Media
- State

2. Core Relationship

The central chain is:



3. ExperimentRun

ExperimentRun is the central entity for a specific experiment run.

It includes, among other things:

- sessionId → association with a Session
- status → for example, created, running, paused, finished, failed, or cancelled
- lifecycle and statistics data
- activeRuleSetId → active RuleSet
- environment or experiment context

Many other models use experimentRunId to associate their data with a particular experiment run.

4. ExperimentParticipant

ExperimentParticipant connects participants to an ExperimentRun, allowing one experiment run to include multiple participants.

Relationship:

ExperimentRun 1 — N ExperimentParticipant

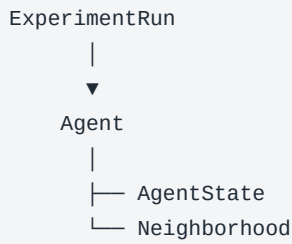
5. Agent

Agent represents an individual device or participant within the crowd/simulation logic.

Important relationships and attributes:

- stateId → AgentState
- neighborhoodId → Neighborhood
- the Agent has position data
- lastSeen and status support online/offline monitoring
- deviceId identifies the Agent

In the proposed architecture, an Agent belongs to exactly **one** Neighborhood.



6. AgentState and AgentStateHistory



This makes it possible to track both an Agent's current state and its historical development.

7. Neighborhood

Neighborhood represents a local area within an experiment.

It includes, among other things:

- `experimentRunId`
- `name`
- `agentIds`
- `configuration`
- global bounds or spatial information

An Agent's position can be interpreted locally within its Neighborhood.

Example: Neighborhood N01

```
1 2 3
4 5 6
7 8 9
```

An Agent might have local coordinates such as:

```
{
  "position": {
    "row": 2,
    "col": 3
  }
}
```

8. NeighborRelation

NeighborRelation describes a direct relationship between two Agents. A relationship can typically be understood as:

Agent A — NeighborRelation — Agent B

In the current architecture concept, an Agent belongs to only one Neighborhood. Relationships between Agents can nevertheless exist within a Neighborhood.

When a failed Agent is replaced, its existing relationships are transferred to the replacement Agent.

9. Signal

Signal represents a message or signal originating from one Agent and potentially addressed to another.

Important relationships:

- sourceAgentId → source Agent
- targetAgentId → target Agent
- experimentRunId → experiment run
- neighborhoodId → local area
- correlationId → correlation of related operations

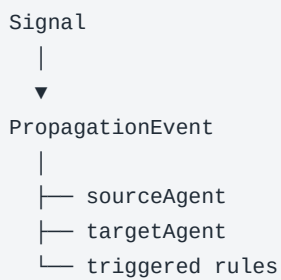
Agent A
|
| Signal
▼
Agent B

10. PropagationEvent

PropagationEvent records what happens to a Signal during forwarding. In particular, it links:

- signalId
- experimentRunId
- neighborhoodId
- sourceAgentId
- targetAgentId
- triggeredRuleIds
- status
- correlationId

Relationship:



Thus, Signal describes the message, while PropagationEvent describes its forwarding or processing.

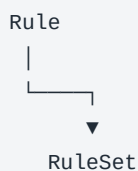
11. Rule, RuleSet, and Threshold

Rule describes an individual rule.

RuleSet groups multiple rules into a configuration.

Threshold can represent thresholds for decision-making or behavioral logic.

Simplified relationship:

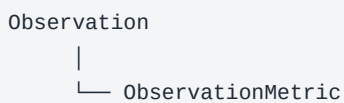




The `ExperimentRun` can refer to the currently active rule set through `activeRuleSetId`.

12. Observation and ObservationMetric

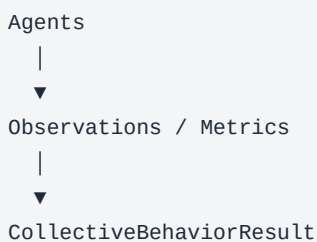
`Observation` stores an observed situation or event. `ObservationMetric` stores measurements or metrics associated with an observation.



This allows the raw observation and its stored measurements to be handled separately.

13. CollectiveBehaviorResult

`CollectiveBehaviorResult` stores the results of an analysis of collective behavior. It belongs primarily to the results/analytics layer rather than the level of an individual technical event.



14. ProtocolEvent

`ProtocolEvent` is the central log of important domain and technical processes.

Examples from the existing routes:

- `SESSION_CREATED`
- `SESSION_CONNECTED`
- `SESSION_DISCONNECTED`

- SIGNAL_CREATED
- SIGNAL_STATUS_CHANGED
- PROPAGATION_CREATED
- PROPAGATION_STATUS_CHANGED
- NEIGHBORHOOD_CREATED
- NEIGHBORHOOD_UPDATED
- RULESET_CREATED
- RULESET_ASSIGNED
- AGENT_REPLACED

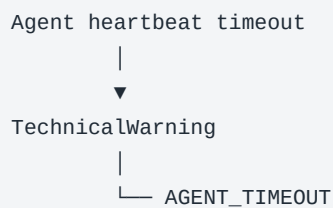
Depending on the event, a `ProtocolEvent` can reference, for example:

- `experimentRunId`
- `agentId`
- `sessionId`
- `signalId`
- `propagationEventId`

15. TechnicalWarning

`TechnicalWarning` describes technical problems or warnings.

Example:

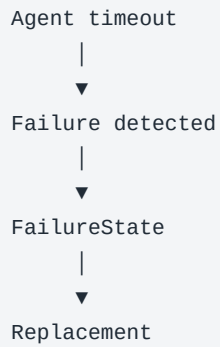


This model is intended for technical warnings and should not replace normal domain-event logging.

16. FailureState

`FailureState` represents an actual error or failure state.

Example:



A `TechnicalWarning` can indicate a problem, whereas `FailureState` represents the actual failure condition.

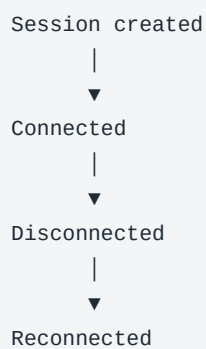
17. Session

`Session` is a proposed concept for a technical connection or session; it is not a local model in the current Local Intelligence runtime.

The broader Crowds model sketch proposes storing, among other things:

- `type`
- `socketId`
- `performerId`
- `isConnected`
- `lastSeen`
- `disconnectReason`

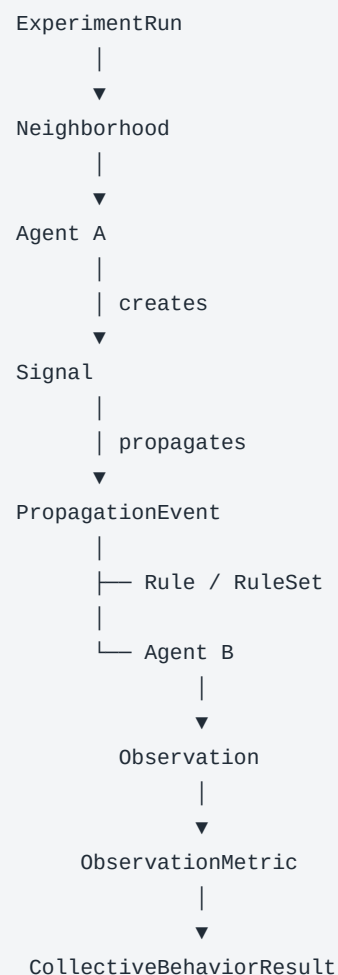
Lifecycle:



The proposed design also logs corresponding state changes through `ProtocolEvent`; this Session lifecycle is not implemented by the current Local Intelligence API.

18. Example of an End-to-End Flow

One possible flow is:



Important system events are logged in parallel:



19. The Three Most Important Event Concepts

Concept	Question it answers
ProtocolEvent	What happened in the system?
TechnicalWarning	What technical problem or warning signal was detected?
FailureState	What actual error or failure condition exists?

Example:

```
Agent does not respond
|
├── TechnicalWarning
|   "AGENT_TIMEOUT"
|
├── FailureState
|   "Agent failed"
|
└── ProtocolEvent
    "AGENT_REPLACED"
```

Summary

The models are interconnected and form a cohesive data structure:

- ExperimentRun defines the experiment context.
- ExperimentParticipant connects participants to the experiment.
- Agent represents devices/participants.
- Neighborhood organizes Agents into local areas.
- NeighborRelation describes Agent-to-Agent relationships.
- AgentState and AgentStateHistory manage current state and history.
- Signal represents communication.
- PropagationEvent represents the processing/forwarding of a Signal.
- Rule, RuleSet, and Threshold support behavioral logic.
- Observation and ObservationMetric store observations and measurements.
- CollectiveBehaviorResult stores analysis results.

- `ProtocolEvent` logs important system events.
- `TechnicalWarning` logs technical warnings.
- `FailureState` describes actual failure conditions.
- `Session` manages technical connections.
- `State` and `Media` support media and state data.

The workspace implementations of these concepts include [ExperimentRun](#), [ExperimentParticipant](#), [Agent](#), [AgentState](#), [AgentStateHistory](#), [Neighborhood](#), [NeighborRelation](#), [Signal](#), [PropagationEvent](#), [Rule](#), [RuleSet](#), [Threshold](#), [Observation](#), [ObservationMetric](#), [CollectiveBehaviorResult](#), [ProtocolEvent](#), [TechnicalWarning](#), [FailureState](#), and [State](#).

Local Intelligence Data Model Proposal

v0.1

Status: Proposal (version 0.1). This document presents a proposed data model for discussion and further development; it is not, by itself, a claim that every field or relationship is implemented in the current backend.

Agent

Connection to Crowds

The existing Crowds backend already manages connected performer devices, active sessions, and real-time communication through Socket.IO.

An Agent extends this runtime by representing a performer as an autonomous participant in a Local Intelligence experiment. Rather than introducing a new client type, Agent provides a logical abstraction that can maintain its own state, exchange signals, and participate in local interactions.

Fields

Field	Type	Description
<code>id</code>	string	Unique identifier of the Agent
<code>deviceId</code>	string	Identifier of the connected Crowds

Field	Type	Description
		device
stateId	AgentState	Current state of the Agent
neighborhoodId	string	Neighborhood to which the Agent belongs

Relationships

- belongs to one Neighborhood
- has one current AgentState
- can send and receive Signals
- participates in one ExperimentRun through its Neighborhood
- may be connected to multiple NeighborRelations

Example Record

Readable example:

```
{
  "id": "agent-17",
  "deviceId": "phone-17",
  "stateId": "state-19",
  "neighborhoodId": "7"
}
```

AgentState

Connection to Crowds

The Crowds backend already stores global application state through `State.js`, allowing the system to synchronize the overall runtime of a session.

AgentState introduces a separate layer for modeling the internal state of an individual Agent during a Local Intelligence experiment. While the existing backend focuses on global synchronization, AgentState enables each Agent to change its behavior independently in response to local interactions and rules.

Fields

Field	Type	Description
id	string	Unique identifier
agentId	string	Reference to the Agent
state	string (enum)	Current state (inactive, waiting, active, ...)
updatedAt	datetime	Time of the last state change

Relationships

- belongs to one Agent
- may be updated by one or more Rules
- may be recorded during an ExperimentRun

Example Record

Readable example:

```
{
  "id": "state-19",
  "agentId": "agent-17",
  "state": "active",
  "updatedAt": "2026-06-25T14:23:41Z"
}
```

Neighborhood

Connection to Crowds

The current Crowds backend manages sessions and connected performers but has no concept of local interaction groups.

Neighborhood introduces this additional abstraction by grouping Agents within an ExperimentRun. It defines the local interaction context in which Signals may propagate and Rules are evaluated. Neighborhood membership may be derived from existing session information, seating layouts, or experiment-specific configurations.

Fields

Field	Type	Description
id	string	Unique identifier
experimentRunId	string	Reference to the ExperimentRun
bounds	Object	Global position of the Neighborhood (rowStart, rowEnd, colStart, colEnd), inclusive and 1-indexed. Used only to derive adjacency.
agentIds	string []	References to Agents belonging to the Neighborhood. An Agent belongs to exactly one Neighborhood.
neighbors	Object []	Adjacent Neighborhoods, each represented by (neighborhoodId, direction, distance)

Possible Future Field

name (string) — Optional human-readable identifier for operators. This field is not required by the core model but could improve usability during experiment setup and monitoring.

Relationships

- belongs to one ExperimentRun
- contains multiple Agents
- contains multiple NeighborRelations
- may receive Signals during an ExperimentRun

Example Record

Readable example:

```
{
  "id": "7",
  "experimentRunId": "12",
```

```
"bounds": { "rowStart": 1, "rowEnd": 3, "colStart": 4, "colEnd": 6 },
"agentIds": [
  "agent-17",
  "agent-18",
  "agent-19",
  "agent-20"
],
"neighbors": [
  { "neighborhoodId": "6", "direction": "west", "distance": 3 },
  { "neighborhoodId": "8", "direction": "east", "distance": 3 }
]
}
```

Signal

Connection to Crowds

The existing Crowds backend already distributes synchronized events between the controller, performers, and visitors through Socket.IO.

Signal builds on this communication infrastructure by representing experiment-specific information exchanged within a Local Intelligence experiment. Signals may originate from an ExperimentRun or an Agent, allowing local interactions, rule evaluation, and signal propagation to be modeled independently of the underlying transport mechanism.

Fields

Field	Type	Description
id	string	Unique identifier
type	string	Type of signal
sourceId	string	Origin of the Signal (Agent or Experiment run)
targetId	string	Target Agent or Neighborhood
payload	Object	Signal-specific data
timestamp	datetime	Time the Signal was created

Relationships

- may originate from an ExperimentRun
- may originate from an Agent
- may target a Neighborhood or an Agent
- may trigger one or more Rules
- may generate multiple PropagationEvents

Example Record

Readable example:

```
{
  "id": "501",
  "type": "pulse",
  "sourceId": "agent-17",
  "targetId": "agent-18",
  "payload": {
    "strength": 1.0,
    "color": "red"
  },
  "timestamp": "2026-06-25T19:30:00Z"
}
```

RuleSet

Connection to Crowds

A RuleSet defines the behavioral logic that can be applied during a Crowds session. The existing backend has no explicit rule system; instead, real-time behavior is handled through Socket.IO events (controller → performer/visitor).

RuleSets introduce a structured abstraction layer that interprets incoming Socket.IO signals and translates them into Agent-level behavior changes, such as AgentState updates and Signal propagation.

Fields

Field	Type	Description
id	string	Unique identifier
experimentRunI	string	Reference to the ExperimentRun

Field	Type	Description
d		
ruleIds	string[]	List of behavioral rules
active	boolean	Whether the RuleSet is currently active

Possible Future Field

name (string) — Optional human-readable identifier for operators. This field is not required by the core model but could improve usability during experiment setup and monitoring.

Relationships

- belongs to one ExperimentRun
- references multiple Rules (via ruleIds)
- evaluates Signals
- can update AgentState

Example Record

Readable example:

```
{
  "id": "ruleset-1",
  "experimentRunId": "12",
  "active": true,
  "ruleIds": [
    "rule-1",
    "rule-2"
  ]
}
```

Rule

Connection to Crowds

Rules define how incoming Socket.IO events (controller/performer/visitor updates) are interpreted in a Local Intelligence context. In the Crowds backend, this logic currently lives implicitly in socket handlers (`controller.js`, `performer.js`). Rules make it explicit and configurable.

Fields

Field	Type	Description
id	string	Unique identifier
type	string	Rule type (e.g. threshold, pattern-match)
signalType	string	Type of signal to which the Rule reacts
action	string	Action to execute (activate, propagate, sync)
threshold	number	Activation threshold

Relationships

- belongs to a RuleSet
- evaluates a Signal
- may update AgentState
- may generate a PropagationEvent

Example Record

Readable example:

```
{
  "id": "rule-1",
  "type": "threshold",
  "signalType": "pulse",
  "action": "activate",
```

```
"threshold": 2
}
```

ExperimentRun

Connection to Crowds

An ExperimentRun maps almost one-to-one to a Crowds “session.” In the backend, a session is already represented by `Session.js` and coordinated through Socket.IO namespaces.

ExperimentRun extends this concept by adding:

- Agent-level modeling
- Neighborhood structure
- rule execution
- an observability layer

Fields

Field	Type	Description
id	string	Unique identifier
sessionId	string	Reference to a Crowds Session
status	string	running/paused/finished
startedAt	datetime	Start time
endedAt	datetime	End time

Relationships

- uses one Crowds Session (`Session.js`)
- contains multiple Neighborhoods

- contains one RuleSet
- produces ObservationMetrics
- produces CollectiveBehaviorResults

Example Record

Readable example:

```
{
  "id": "exp-1",
  "sessionId": "1",
  "status": "running",
  "startedAt": "2026-06-25T19:00:00Z",
  "endedAt": null
}
```

PropagationEvent

Connection to Crowds

PropagationEvents represent what currently happens implicitly in Socket.IO broadcasts:

```
controller emits event → backend forwards → performer receives
```

In Local Intelligence terms, each broadcast becomes a traceable propagation step. This is especially useful because the backend already has real-time event flows through:

- /controller
- /performer
- /visitor

Fields

Field	Type	Description
id	string	Unique identifier
signalId	string	Related Signal
sourceId	string	Source of the Signal

Field	Type	Description
targetId	string	Target of the Signal
timestamp	datetime	Time of propagation
status	string	success/blocked/delayed

Relationships

- belongs to a Signal
- connects two Agents
- may trigger Rule evaluation
- is part of ExperimentRun tracing

Example Record

Readable example:

```
{
  "id": "prop-88",
  "signalId": "501",
  "sourceId": "agent-17",
  "targetId": "agent-18",
  "timestamp": "2026-06-25T19:30:01Z",
  "status": "success"
}
```

ObservationMetric

Connection to Crowds

The Crowds backend already exposes `/metrics` (in-memory counters for connections and messages). ObservationMetrics extend this into experiment-level analytics. Instead of tracking only system health, the model measures collective behavior patterns.

Fields

Field	Type	Description
id	string	Unique identifier
experimentRunId	string	Reference to the ExperimentRun
type	string	Metric type (sync, latency, clustering)
value	number	Measured value
timestamp	datetime	Measurement time

Relationships

- belongs to an ExperimentRun
- computed from PropagationEvents
- derived from AgentState changes
- used to evaluate CollectiveBehaviorResult

Example Record

Readable example:

```
{
  "id": "metric-1",
  "experimentRunId": "exp-1",
  "type": "synchronization-score",
  "value": 0.87,
  "timestamp": "2026-06-25T19:35:00Z"
}
```

CollectiveBehaviorResult

Connection to Crowds

CollectiveBehaviorResult represents the interpreted outcome of a Local Intelligence ExperimentRun. While the Crowds backend currently focuses on real-time synchronization and system health through `/metrics`, this entity introduces a semantic interpretation layer describing emergent group behavior.

It is computed from:

- ObservationMetrics (quantitative signals such as sync, latency, and clustering)
- PropagationEvents (how Signals moved through the system)
- AgentState transitions (how Agents reacted over time)

In the existing Crowds architecture, this would not be stored directly today, but could be derived after a session from MongoDB session logs and Socket.IO event traces.

Fields

Field	Type	Description
id	string	Unique identifier
experimentRunId	string	Reference to the ExperimentRun
type	string	Type of emergent behavior (sync, clustering, wave, consensus)
score	number	Normalized strength of the observed behavior (0–1)
confidence	number	Confidence in the measurement based on data completeness
startTime	datetime	Start time
endTime	datetime	End time
summary	string	Human-readable interpretation of the result

Field	Type	Description
-------	------	-------------

Relationships

- belongs to one ExperimentRun
- derived from ObservationMetrics
- derived from PropagationEvents
- derived from AgentState changes
- summarizes global system behavior across a Neighborhood or a full ExperimentRun

Example Record

Readable example:

```
{
  "id": "cbr-1",
  "experimentRunId": "exp-1",
  "type": "synchronization",
  "score": 0.91,
  "confidence": 0.84,
  "startTime": "2026-06-25T19:20:00Z",
  "endTime": "2026-06-25T19:35:00Z",
  "summary": "High synchronization emerged across all agents after repeated pulse signals propagated through the network."
}
```

Existing Backend → Local Intelligence Mapping

Crowds Backend	Local Intelligence Concept
Session.js	ExperimentRun
State.js	AgentState
Socket.IO events	Signal
controller/performer/visitor sockets	Agent interactions

Crowds Backend	Local Intelligence Concept
/metrics endpoint	ObservationMetric (system-level baseline)
MongoDB session storage	ExperimentRun + Neighborhood persistence
real-time broadcasts	PropagationEvent

The identifiers and field names above are retained as proposed in v0.1. The implementation may differ; consult the corresponding workspace models, including [Agent](#), [AgentState](#), [Neighborhood](#), [Signal](#), [Rule](#), [RuleSet](#), [ExperimentRun](#), [PropagationEvent](#), [ObservationMetric](#), and [CollectiveBehaviorResult](#).

Local Intelligence entity map (v0.1)

This is a high-level conceptual map of the prototype's domain. The relationships below explain the domain; they do not imply that every association is enforced as a database foreign key or stored in a separate collection.

Entity overview

- **Experiment:** ExperimentRun provides the context for a run; ExperimentParticipant associates participants with it.
- **Agents and neighborhoods:** Agent, AgentState, AgentStateHistory, Neighborhood, and NeighborhoodRelation describe participants, their current and historical state, and local connections. NeighborhoodRelation is the implementation's model name; the source notes also call it NeighborRelation.
- **Signals and rules:** Signal and PropagationEvent describe messages and their propagation; RuleSet, Rule, and Threshold describe behavioral configuration.
- **Observation and results:** Observation, ObservationMetric, and CollectiveBehaviorResult cover recorded observations, measures, and analysis outcomes.
- **Operations and diagnostics:** ProtocolEvent, TechnicalWarning, and FailureState represent event records, technical warnings, and failure conditions.
- **Supporting model:** State defines state data used by the prototype.

Conceptual relationships

```
ExperimentRun
├─ ExperimentParticipant
├─ Agent — AgentState / AgentStateHistory
│   └─ Neighborhood — NeighborhoodRelation — Agent
│       └─ Signal — PropagationEvent
├─ RuleSet — Rule — Threshold
├─ Observation — ObservationMetric
├─ CollectiveBehaviorResult
├─ ProtocolEvent
├─ TechnicalWarning
└─ FailureState
```

An agent's state and neighborhood membership are modeled separately. Signals may be created by an agent or by experiment-level control logic; propagation and rule evaluation depend on the experiment configuration.

Implementation status and open questions

The repository exports model schemas for Agent, AgentState, AgentStateHistory, CollectiveBehaviorResult, ExperimentParticipant, ExperimentRun, FailureState, Neighborhood, NeighborhoodRelation, Observation, ObservationMetric, PropagationEvent, ProtocolEvent, Rule, RuleSet, Signal, State, TechnicalWarning, and Threshold from the [model index](#). This confirms that model definitions exist; it does not by itself mean every workflow or relationship is fully implemented or persisted through those individual models. Experiment-run persistence also stores a run snapshot.

Session and Media appear in the broader source model sketch, but there are no corresponding exported model schemas in the current model index. Treat them as proposed or external concepts, not as implemented local models. In particular, `ExperimentRun.sessionId` is an optional external session identifier, not a reference to a local Session model.

The original map asks whether experiment-level and agent-to-agent communication should share one Signal entity. That remains a modeling question; this map does not settle it.

Signal-flow examples

```
Agent
↓
Signal
↓
NeighborhoodRelation
↓
```

```
Agent
↓
Rule
↓
AgentState

ExperimentRun
↓
Signal
↓
Neighborhood
↓
Agent
↓
Signal
↓
NeighborhoodRelation
↓
Agent
↓
Rule
↓
AgentState
```

See also [Prototype database and data model](#) and [Signal, autonomy, and follow-up example](#).

API Routes

This document records a proposed route inventory and the behavior associated with each route. It is a specification, not confirmation that these endpoints or features are implemented in the current codebase. Verify each route against the running application and source before treating it as available.

Current implementation note: The routes in this proposal are not mounted by the current Local Intelligence app unless independently verified in its source. Session, Media, Simulator, Analytics, and Recovery routes below are design proposals; their MongoDB-backed behavior is not implemented by the current run-snapshot persistence layer.

The endpoint strings below are preserved exactly as supplied. In particular, PATCH//experiments/:id/complete and GET//experiments/:id/export contain doubled slashes in the source; whether those slashes are intentional is uncertain and should be verified.

Experiments

- `POST /experiments` — Creates an experiment run, initializes its lifecycle, links it to a session, and sets the experiment status to running.
- `PATCH//experiments/:id/complete` — Completes an experiment run, records the end time, updates the final status, summary, and result, and logs a protocol event.
- `GET /experiments/:id/summary` — Returns a statistical summary of an experiment, including participants, signals, propagation events, observations, warnings, failures, and collective behavior results.
- `GET//experiments/:id/export` — Exports experiment-related data, including agents, participants, signals, propagation events, observations, protocol events, warnings, failures, and analysis results.
- `GET /experiments/:id/timeline` — Returns the chronological sequence of protocol events recorded during an experiment.

Agents and neighborhoods

- `POST /agents` — Registers a mobile device as an agent, creates its initial state, and marks the agent as online.
- `PATCH /agents/:id/state` — Updates an agent's current state, stores the transition in the history, and records the change as a protocol event.
- `POST /neighborhoods` — Creates a neighborhood for an experiment and assigns the selected agents to it.

Signals and propagation

- `POST /signals` — Creates a signal generated by an agent, assigns a correlation ID, and logs the signal creation.
- `PATCH /signals/:id/status` — Updates a signal's processing status and records the status change in the protocol log.
- `POST /propagation` — Creates a propagation event representing the transmission of a signal between two agents and records the event.
- `PATCH /propagation/:id/status` — Updates a propagation event's status (for example, sent, received, processed, or failed) and records the transition.

Diagnostics and authentication

- `GET /diagnostics` — Provides an overview of current system health, including agent status, running experiments, and recent activity during the last 30 minutes.
- `GET /experiments/:id/diagnostics` — Returns diagnostic information for a specific experiment, including statistics, warnings, failures, recent protocol events, and behavior-analysis results.
- `POST /auth` — Authenticates the controller using a password, applies rate limiting to failed attempts, and returns a JWT valid for 24 hours.

Analytics

- `GET /analytics/events` — Returns the latest persistent protocol events from the database, with an optional `limit` parameter controlling the number of returned events.
- `GET /analytics/summary` — Returns an analytics summary for a specified time range, including the total number of protocol events and their distribution by event type and namespace.

Rule sets

- `POST /rulesets` — Creates a RuleSet for an existing experiment, stores its rules and configuration, and records its creation as a protocol event.
- `PATCH /rulesets/:id/assign` — Assigns an existing RuleSet to an experiment, makes it the experiment's active RuleSet, activates the RuleSet, and records the assignment as a protocol event.

Sessions

- `POST /sessions` — Creates a session for a connected client or performer, stores its socket and performer information, and records its creation as a protocol event.
- `GET /sessions/:id` — Returns a session by its ID, including its connection status and associated information.
- `PATCH /sessions/:id/disconnect` — Disconnects a session, stores the disconnection reason and last-seen time, and records the disconnection as a protocol event.
- `PATCH /sessions/:id/connect` — Reconnects a session, clears the previous disconnection reason, updates the last-seen time, and records the reconnection as a protocol event.

Simulator

- `GET /simulator/scenarios` — Returns the available simulator scenarios that can be selected for a run.
- `GET /simulator/runs` — Returns a list of simulator runs that have been created or executed.
- `DELETE /simulator/reports` — Deletes or clears generated simulator reports and returns the cleanup result.
- `POST /simulator/runs` — Starts a simulator run for a selected scenario and returns the newly created run.
- `GET /simulator/runs/:runId` — Returns the current information and status of a specific simulator run.
- `GET /simulator/runs/:runId/report` — Returns the generated report for a completed simulator run by reading the Markdown report and rendering it as an HTML document.

Media

- `GET /visuals` — Returns the available visual media files and parsed metadata, using MongoDB data when available and falling back to the `public/visuals` directory.
- `GET /sounds` — Returns the available sound files and parsed metadata, using MongoDB data when available and falling back to the `public/sounds` directory.
- `GET /mp3s` — Returns the available MP3 files and parsed metadata, using MongoDB data when available and falling back to the `public/mp3s` directory.

Recovery

- `GET /recovery/database` — Returns the current MongoDB load, including database statistics, collection count, storage usage, and document counts for sessions, states, and media.
- `DELETE /recovery/database/sessions` — Deletes disconnected sessions from the database and returns the number deleted together with the updated database load.

Related documentation

- [Overview and API Routes](#)
- [Event Taxonomy and Schema Governance](#)

Data-Layer Handover: Models and API Routes

This document summarizes conceptual data models and proposed route capabilities for the Crowds Local Intelligence / Data Layer. Model relationships and route behavior below reflect the documented architecture and intended design; they are not evidence that every model, route, or feature exists in the current codebase. Verify implementation separately before relying on any item as available. See the [route specification](#) for the detailed proposal.

1. Data models

The Crowds Local Intelligence / Data Layer design describes data models for experiments, agents, neighborhoods, signals, propagation, observations, and technical events.

Central models

- **ExperimentRun** — Represents one experiment run. It includes information such as status, environment, and runtime details, and serves as an overarching link for many other records.
- **ExperimentParticipant** — Links agents or other participants to an ExperimentRun.
- **Agent** — Represents a device or participant in the system. An agent may have status, position, and a neighborhoodId, among other properties.
- **AgentState / AgentStateHistory** — Store an agent's current state and its historical state changes.
- **Neighborhood** — Describes a local area within an experiment and may contain associated agents. The intended architecture assigns an agent to one neighborhood, but assignment is optional in the current schema.
- **NeighborRelation** — Describes relationships between agents within or across local areas. The identifier is retained as written in the source document.
- **Signal** — Stores a signal created by one agent and potentially passed to another. The record includes properties such as source, target, status, and correlationId.
- **PropagationEvent** — Documents the forwarding or processing of a signal between agents, making the signal's path traceable.
- **Observation / ObservationMetric** — Store observations and their associated measurements during an experiment.

- **Rule / RuleSet / Threshold** — Represent rules and configurations used for processing and behavior within an experiment.
- **ProtocolEvent** — Provides central event logging. Important actions, such as creating a signal, changing its status, or creating a neighborhood, are intended to be recorded as events.
- **TechnicalWarning / FailureState** — Document technical problems, warnings, and failure states, for example an agent timeout or outage.
- **Session** — A proposed or external concept for a technical connection. The current Local Intelligence model index does not export a local Session model.

Model relationships

The central relationship can be simplified as:

ExperimentRun → Neighborhood → Agent

Communication can be represented as:

Agent → Signal → PropagationEvent → Agent

Important processes are additionally documented through ProtocolEvent; technical problems are recorded through TechnicalWarning or FailureState.

Together, these relationships describe a traceable data structure for storing system state, communication, and experiment events. They describe the intended data model and do not certify that every relationship is enforced by the current implementation.

2. API routes

The routes below describe the interface envisioned between the frontend or simulator and the database. They cover creating, updating, retrieving, and, in some cases, deleting data. Treat this section as a route and behavior specification, not as a verified inventory of implemented endpoints.

Experiment and lifecycle routes

The experiment-related routes are intended to manage experiment runs and their lifecycle.

The documented cleanup route is:

POST /lifecycle/cleanup/test-experiments

It is intended to clean up old test experiments and associated data, including participants, signals, propagation events, observations, and technical events. Its current implementation is not confirmed here.

Neighborhood routes

- `POST /neighborhoods` — Creates a neighborhood and may assign agents to it directly.
- `PATCH /neighborhoods/:id` — Updates properties such as the name, configuration, or agent assignment.

The described behavior checks whether agents exist and whether they are already assigned to another neighborhood.

Signal routes

- `POST /signals` — Creates a signal and checks, among other things, that the source agent and any optional target agent exist.
- `PATCH /signals/:id/status` — Changes a signal's status, for example:

`created` → `sent` → `received` → `processed`

The status change is also intended to be documented as a `ProtocolEvent`.

Propagation routes

- `POST /propagation` — Creates a `PropagationEvent` for forwarding a signal.
- `PATCH /propagation/:id/status` — Updates the propagation's processing status and may store timestamps for received and processed.

These changes are also intended to be logged through `ProtocolEvent`.

Session routes

- `POST /sessions` — Creates a session.
- `GET /sessions/:id` — Retrieves a specific session.
- `PATCH /sessions/:id/connect` — Marks a session as connected.
- `PATCH /sessions/:id/disconnect` — Marks a session as disconnected and stores the reason.

The major session-lifecycle changes are also intended to be recorded as `ProtocolEvent` entries.

RuleSet routes

- `POST /rulesets` — Creates a RuleSet for an experiment.
- `PATCH /rulesets/:id/assign` — Assigns a RuleSet to an experiment and makes it the active RuleSet.

Media routes

The media routes are intended to provide media available to the system:

- `GET /visuals`
- `GET /sounds`
- `GET /mp3s`

The described design prefers data from MongoDB. If no list is available there, it falls back to files in the `public` directory. This behavior is not confirmed as implemented.

Recovery and simulator capabilities

The described recovery capabilities include monitoring MongoDB load and deleting disconnected sessions.

The described simulator capabilities include:

- Listing available test scenarios.
- Starting simulator runs.
- Retrieving running runs.
- Displaying reports.
- Deleting old simulator reports.

The source document does not specify route paths for these recovery and simulator capabilities; none are inferred here.

Summary

The documented API design covers the following lifecycle:

Create → Update → Read → Lifecycle → Logging → Cleanup

A key design principle is that relevant system actions should not only change the underlying data but also be traceable through `ProtocolEvent`. This is intended to support later analysis, monitoring, debugging, and KPI evaluation; it should not be read as confirmation that those capabilities are currently implemented.

3. Priority 8 — Interactive / Taiwan Observability Support

According to the source handover note, only “Rehearsal/test-run grouping” was completed. The other items are unfinished and need revision. This status statement has not been independently verified.

Related documentation

- [Event Taxonomy and Schema Governance](#)
- [API Routes](#)

Event Taxonomy and Schema Governance v0.1

This document defines a unified event taxonomy and schema-governance strategy for the Crowds ecosystem.

Event naming convention

Event names use the `DOMAIN_ACTION` format: uppercase words separated by underscores, with the verb at the end.

Examples:

- `SIGNAL_CREATED`
- `SIGNAL_STATUS_CHANGED`
- `PROPAGATION_CREATED`
- `AGENT_STATE_CHANGED`
- `EXPERIMENT_COMPLETED`

Governance rules

All new events in this project must follow these rules:

1. **Use the standard event-name format.** Event names must follow DOMAIN_ACTION.
2. **Assign exactly one category to each event.** The permitted categories are:
 - runtime
 - protocol
 - experiment
 - analytics
3. **Include the required fields in every ProtocolEvent document:**
 - type
 - category
 - schemaVersion
 - timestamp
 - experimentRunId
 - agentId
 - signalId
 - propagationEventId
 - correlationId
 - detail
4. **Increment schemaVersion** whenever the event payload changes incompatibly.
5. **Before adding an event:**
 - Validate all referenced IDs.
 - Use standardized event names.
 - Update the documentation whenever a new event type is added.

This specification is intended to keep Crowds, Grid, Interactive, Local Intelligence, and future research systems compatible. It describes the governance requirements; it does not by itself confirm that every event or field is implemented in the current codebase.

Related documentation

- [Overview and API Routes](#)

- [API Routes](#)

Tick-Based Simulation

Motivation

The current runtime processes signals and can also run scheduled simulation ticks while an experiment is active. However, the existing tick mechanism is not yet the fully configurable, per-agent loop proposed below.

A simulation engine that periodically updates all agents could enable continuous adaptation and more realistic collective behavior. This is a proposed future improvement, not behavior provided by the current runtime.

Current flow

Signal received

↓

`processSignal()`

↓

Rule evaluation

↓

Swarm behavior

↓

Logging

Proposed flow

Experiment run starts

↓

Simulation loop

↓

For each simulation tick

↓

For each agent

↓

Analyze neighborhood

↓

Evaluate rules

↓

Execute swarm behavior

↓

Generate signals if needed

↓

Logging

↓

Next tick

Potential benefits

- Continuous adaptation
- More realistic swarm behavior
- Time for emergent behavior to develop
- Self-organization without external events
- A foundation for implementing future algorithms

Signal processing

In the proposed design, signals would no longer be the system's primary trigger. Instead, they would be events processed during a simulation tick.

This separates:

- Simulation timing
- Rule evaluation
- Signal propagation

Possible configuration

```
{
  "tickRate": 100,
  "maxTicks": 10000,
  "autoStop": true
}
```

Possible future extensions

- Variable tick rate
- Pause and resume
- Distributed simulation
- Parallel neighborhood processing
- Priority scheduling
- Dynamic agent updates

Status and notes

This document proposes extending the existing tick mechanism. The current scheduler invokes a simulation tick every 1,000 ms by default while a run is active; the richer tick-rate, per-agent processing, and scheduling options described above are proposals, not all current runtime behavior. Any extension should preserve existing behavior and be evaluated against the data model and persistence design.

Proposal: Intelligent Signal Routing for the Local Intelligence Layer

Status: Proposal. This document describes a possible future capability; it does not claim that intelligent routing is already implemented.

Motivation

The current architecture provides a strong foundation:

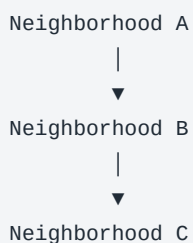
- Each Agent can be assigned to at most one Neighborhood; assignment is optional in the current schema.
- Neighborhoods form an explicit graph through Neighborhood-to-Neighborhood connections.
- Signals can propagate between connected Neighborhoods.

The next logical step is to make signal propagation **intelligent**. Rather than simply forwarding a Signal to every reachable Neighbor, a Signal could select an efficient route to its destination. This would transform propagation from simple flooding into goal-oriented distributed routing.

Current Situation

The current propagation engine forwards a Signal from one Agent to neighboring Agents. When a Signal reaches a Neighborhood boundary, it may continue into one of the connected Neighborhoods.

Conceptually:



This approach works well for local propagation but can become inefficient as an experiment grows.

The Problem

Imagine an experiment consisting of dozens or hundreds of Neighborhoods. A Signal starts inside **Neighborhood 1** and is intended for **Neighborhood 10**.

Without routing intelligence, the Signal has no knowledge of the destination and propagates step by step through the graph. This can lead to:

- unnecessary propagation
- duplicated work
- additional latency
- increased CPU usage
- unnecessary memory consumption
- excessive propagation events

As the experiment scales, this approach becomes increasingly inefficient.

Routing Targets

A Signal should be able to specify its destination. Two destination types are useful.

Target Neighborhood

```
{
  "targetNeighborhoodId": "NH10"
}
```

The objective is to deliver the Signal to a specific Neighborhood.

Target Agent

```
{
  "targetAgentId": "agent-275"
}
```

The system first determines which Neighborhood currently owns the Agent:

```
Agent
  ↓
Neighborhood
```

↓
Routing

When an Agent is assigned to a Neighborhood, its neighborhood can be looked up from that assignment.

Why Agent Lookup Is Not Expensive

At first glance, searching for an Agent before routing may seem like an expensive extra operation. In practice, it is usually negligible:

Agent ID
↓
Lookup Neighborhood
↓
Calculate Route

With a hash map, Agent-ID lookup is typically $O(1)$ on average. Route-computation cost depends on graph size and the selected algorithm; it is not necessarily more expensive than every Agent lookup. Supporting both destination types adds an Agent-to-Neighborhood lookup when the target is an Agent.

Intelligent Routing

Instead of forwarding blindly, the Signal could use a route through the Neighborhood graph. For example:

```
NH01
|
▼
NH04
|
▼
NH08
|
▼
NH10
```

The calculated route could be stored inside the Signal:

```
{
  "route": [
    "NH01",
    "NH04",
    "NH08",
    "NH10"
  ]
}
```

```
]
}
```

During propagation, the Signal would follow the planned route.

Multiple Routing Strategies

Different experiments may require different routing behavior. Possible strategies include:

- shortest path
- lowest latency
- lowest propagation cost
- highest bandwidth
- highest reliability
- minimum hop count

The routing strategy could therefore become part of the Signal:

```
{
  "routingStrategy": "lowestLatency"
}
```

Connection Metadata

Neighborhood connections can contain additional information, for example:

```
{
  "targetNeighborhoodId": "NH05",
  "latency": 12,
  "bandwidth": 100,
  "packetLoss": 0.01,
  "cost": 2
}
```

Routing decisions could use this connection quality instead of treating every connection equally.

Dynamic Route Recalculation

Experiments are dynamic, and connections may change while a Signal is travelling. Examples include:

- a Neighborhood becoming unavailable
- a server going offline
- high latency
- congestion
- a temporary connection failure

Rather than failing immediately, the Signal could calculate a new route from its current position.

Future Distributed Routing

A more advanced version could avoid calculating the complete path at the beginning. Instead, each Neighborhood would choose only the next hop based on local information:

```
Signal
  |
  ▼
Neighborhood A
  |
  chooses B
  ▼
Neighborhood B
  |
  chooses D
  ▼
Neighborhood D
```

This resembles routing in distributed computer networks and could enable adaptive behavior.

Possible Signal Extensions

```
{
  "targetNeighborhoodId": "...",
  "targetAgentId": "...",
  "route": [],
  "currentHop": 0,
  "routingStrategy": "shortest",
  "routeCalculated": true
}
```

Possible Neighborhood Extensions

```
{
  "neighbors": {
    "north": "...",
    "south": "...",
    "east": "...",
    "west": "..."
  },
  "connections": [
    {
      "targetNeighborhoodId": "...",
      "latency": 15,
      "cost": 2,
      "bandwidth": 100
    }
  ]
}
```

Possible Routing Algorithms

The most appropriate algorithm depends on the available information.

Algorithm	Best suited for
Breadth-First Search (BFS)	Fewest Neighborhood hops
Dijkstra	Weighted routing (latency, cost)
A*	Spatial Neighborhood layouts with coordinates
Bellman-Ford	Dynamic edge costs
Link-State Routing	Large distributed experiments
Reinforcement Learning	Future adaptive swarm routing

For the current prototype:

- BFS is sufficient for shortest-path routing.
- Dijkstra becomes useful once weighted connections (latency, cost, bandwidth) are introduced.

Relationship to the Existing Neighborhood Proposal

This proposal builds directly on the existing Neighborhood architecture, which introduced:

- one Agent per Neighborhood
- explicit Neighborhood ownership
- explicit Neighborhood connections
- graph-based Neighborhood topology

This proposal extends that architecture by enabling Signals to use those Neighborhood connections intelligently. The existing graph could serve as routing infrastructure. The implementation-level [Signal model](#) and [Neighborhood model](#) provide workspace context. The [path-finding module](#) is currently empty; the routing behavior described here is proposed, not implemented.

Long-Term Vision

Intelligent routing could open the door to advanced swarm behavior, including:

- adaptive routing
- congestion avoidance
- self-healing communication
- distributed path discovery
- collaborative routing between Neighborhoods
- decentralized decision making
- load balancing
- predictive routing
- autonomous optimization
- emergent communication patterns

These capabilities could move the Local Intelligence Layer beyond deterministic signal propagation toward a distributed system capable of self-organizing communication.

Expected Benefits

Compared with simple propagation, intelligent routing could offer:

- fewer propagation events

- reduced computational overhead
- lower network traffic
- faster signal delivery
- better scalability
- improved fault tolerance
- more realistic swarm communication
- a foundation for collective intelligence
- a foundation for emergent behavior
- a foundation for future self-organizing algorithms

Autonomy strategy

This document describes the autonomy flow present in the prototype. It is an implementation overview, not a proposal for a future subsystem.

Function flow

```
agentAutonomy()           → decide
executeAutonomousAction() → apply a decision
runAutonomy()             → process eligible agents
autonomyScheduler()       → run one or more simulation ticks
```

Responsibilities

- `agentAutonomy()` evaluates an agent's local neighborhood and returns a decision such as `activate`, `listen`, `wait`, `idle`, or `observe`. Its current decisions are based on the counts and states of local neighbors.
- `runAutonomy()` skips agents that are offline, lack a neighborhood, or have autonomy suspended; it analyzes each eligible agent, records an observation, and passes the decision to the executor.
- `executeAutonomousAction()` applies supported state changes, logs them, and evaluates state-change rules. An activation can also create an `autonomous_activation` signal for an inactive local neighbor.
- `autonomyScheduler()` runs the requested number of simulation ticks. A tick also analyzes signals and processes due scheduled actions; it is not itself a permanent background timer.

The implementation can be explored in the project under `local-intelligence/services/autonomy/` and `local-intelligence/services/simulation/`.

Related documentation

- [Entity map](#)
- [Prototype database and data model](#)
- [Signal, autonomy, and follow-up example](#)

Example: Signal, autonomy, and follow-up propagation

Status: The autonomy and delayed-action mechanisms used here exist in the prototype. The sequence and rule configuration below are illustrative; they are not a claim that this exact end-to-end scenario has been run or that every propagation detail is guaranteed by the current evaluator.

This example describes a possible flow in a 3×3 agent field. It is documentation only and does not modify runtime data or Rule configuration.

Initial state

X means inactive and A means active.

```
+-----+-----+-----+
| 1 X | 2 X | 3 X |
+-----+-----+-----+
| 4 X | 5 A | 6 X |
+-----+-----+-----+
| 7 X | 8 X | 9 X |
+-----+-----+-----+
```

Agent 5 is active; all other agents are inactive.

Participating agents

Agent 5: active sender of the first signals
Agent 2: target of an activation signal
Agent 4: target of an activation signal
Agent 1: becomes active through an autonomy tick
Agent 9: target of Agent 1's follow-up signal

Phase 1: Agent 5 sends activation signals

Agent 5 sends two activate signals:

```
{
  type: "activate",
  sourceAgentId: "agent-5",
  targetAgentId: "agent-2",
  payload: {
    strength: 1,
  },
  properties: {
    ttl: 10,
    propagationMode: "unicast",
    propagationScope: "neighborhood",
  },
}

{
  type: "activate",
  sourceAgentId: "agent-5",
  targetAgentId: "agent-4",
  payload: {
    strength: 1,
  },
  properties: {
    ttl: 10,
    propagationMode: "unicast",
    propagationScope: "neighborhood",
  },
}
```

An illustrative Rule configuration is:

```
{
  id: "rule-activate-agent",
  enabled: true,
  signalType: "activate",
  scope: "global",
  action: "activate",
}
```

```
threshold: 1,  
}
```

If those signals are accepted and the Rule matches, Agents 2 and 4 become active:

```
+-----+-----+-----+  
| 1 X | 2 A | 3 X |  
+-----+-----+-----+  
| 4 A | 5 A | 6 X |  
+-----+-----+-----+  
| 7 X | 8 X | 9 X |  
+-----+-----+-----+
```

Conceptual flow:

```
Agent 5  
-> activate signal to Agent 2  
  -> signalEngine  
    -> neighborhood analysis  
    -> Rule evaluation  
    -> activate Rule  
    -> Agent 2 becomes active  
    -> logging
```

```
Agent 5  
-> activate signal to Agent 4  
  -> signalEngine  
    -> neighborhood analysis  
    -> Rule evaluation  
    -> activate Rule  
    -> Agent 4 becomes active  
    -> logging
```

Phase 2: An autonomy tick activates Agent 1

In the illustrated grid, Agent 1 has two active local neighbors (Agents 2 and 4). The current autonomy logic can therefore return activate with reason `high_local_activity` for an eligible inactive agent.

The decision has this shape:

```
{  
  agentId: "agent-1",  
  action: "activate",  
  reason: "high_local_activity",  
}
```

The tick applies the decision and evaluates state-change Rules:

```
Autonomy tick
-> Agent 1's neighborhood is analyzed
-> agentAutonomy decides: activate
-> executeAutonomousAction
-> Agent 1 changes from inactive to active
-> state_changed triggers are evaluated
```

The resulting field is:

```
+-----+-----+-----+
| 1 A | 2 A | 3 X |
+-----+-----+-----+
| 4 A | 5 A | 6 X |
+-----+-----+-----+
| 7 X | 8 X | 9 X |
+-----+-----+-----+
```

Phase 3: Agent 1 triggers a follow-up Rule

Agent 1 has an illustrative Rule that reacts to the `inactive -> active` state change:

```
{
  id: "rule-agent-1-follow-up",
  enabled: true,
  scope: "agent",
  agentId: "agent-1",
  trigger: {
    type: "state_changed",
    fromState: "inactive",
    toState: "active",
  },
  action: "send_signal",
  appendedSignal: {
    delayMs: 5000,
    targetAgentId: "agent-9",
    signalType: "deactivate",
    signalPayload: {
      strength: 1,
      reason: "agent-1-became-active",
    },
  },
  signalProperties: {
    ttl: 10,
    propagationMode: "broadcast",
    propagationScope: "all",
  },
},
}
```

The state-change handler schedules this action rather than sending the signal immediately:

```
Agent 1 becomes active
-> state_changed trigger matches
-> appendedSignal is stored as pending
-> wait 5000 ms
-> a later simulation tick checks executeAt
-> a signal to Agent 9 is created
```

An internal scheduled-action record may look like this:

```
{
  action: "send_signal",
  sourceAgentId: "agent-1",
  targetAgentId: "agent-9",
  signalType: "deactivate",
  signalPayload: {
    strength: 1,
    reason: "agent-1-became-active",
  },
  signalProperties: {
    ttl: 10,
    propagationMode: "broadcast",
    propagationScope: "all",
  },
  status: "pending",
  executeAt: "2026-09-22T12:00:05.000Z",
}
```

The scheduled action is processed by a simulation tick after its due time; the delay is not implemented as a permanent background timer.

Phase 4: Agent 9 and further receivers become inactive

For the desired result, the active RuleSet needs a Rule that inactivates the receiver and a propagation Rule that forwards the signal. The following examples describe the intended configuration:

Inactivation Rule

```
{
  id: "rule-deactivate-agent",
  enabled: true,
  signalType: "deactivate",
  scope: "global",
  action: "inactivate",
  threshold: 1,
}
```

This Rule is intended to set the receiving agent to `inactive`.

Propagation Rule

```
{
  id: "rule-propagate-deactivate",
  enabled: true,
  signalType: "deactivate",
  scope: "global",
  action: "propagate",
  threshold: 1,
}
```

This Rule is intended to forward the signal to additional eligible agents. Both Rules must be in the active RuleSet. The example assumes that the evaluator processes the inactivation before forwarding at each receiver; confirm ordering, target selection, and termination behavior against the configured propagation implementation.

Intended per-receiver flow:

```
Deactivate signal
-> inactivate Rule
-> receiver becomes inactive
-> propagate Rule
-> signal is sent to further eligible agents
```

After Agent 9 receives and processes the signal, the field should be shown as:

```
+-----+-----+-----+
| 1 A | 2 A | 3 X |
+-----+-----+-----+
| 4 A | 5 A | 6 X |
+-----+-----+-----+
| 7 X | 8 X | 9 X |
+-----+-----+-----+
```

Agent 9 remains inactive in this displayed step; the signal does not cause an additional state change for that agent. Further agents change state only if propagation reaches them and the inactivation Rule applies.

Complete illustrative flow

```
Initial state:
1X 2X 3X
4X 5A 6X
7X 8X 9X
```

1. Agent 5 sends activate to Agent 2.
2. The activate Rule makes Agent 2 active.
3. Agent 5 sends activate to Agent 4.
4. The activate Rule makes Agent 4 active.
5. An autonomy tick decides activate for Agent 1.
6. Agent 1 becomes active.
7. The state_changed inactive -> active trigger matches Agent 1's Rule.
8. The Rule schedules an appendedSignal with delayMs 5000.
9. A later simulation tick creates deactivate from Agent 1 to Agent 9.
10. The deactivate -> inactivate Rule makes Agent 9 inactive.
11. The deactivate -> propagate Rule forwards the signal, if supported by the active configuration.
12. Each further receiver becomes inactive only if it receives the signal and the inactivation Rule matches.

Technical prerequisites and caveats

- Agent 5 must have two valid target agents, and the two signals are separate.
- Agent 1 must actually transition from inactive to active for the state-change trigger to match.
- Agent 1 needs a neighborhood for the illustrated local-neighborhood behavior.
- Agent 9 must exist for createSignal to create the follow-up signal.
- Agents must be reachable under the active propagation mode, scope, and neighborhood configuration.
- The deactivate -> inactivate and deactivate -> propagate behaviors must be supported and configured in the active RuleSet; the example does not certify their exact evaluator order.
- The delayed action is processed by a simulation tick after 5000 ms have elapsed, not by a permanent background timer.

See [Autonomy strategy](#) and [Entity map](#).

Example: Crows and Holy — interactive show activation

Status: Application concept. This example describes a possible experience using the Local Intelligence Layer. It does not establish that smartphone motion detection, signal orchestration, or show visualization is already implemented.

Goal

Crows could use the Local Intelligence Layer for a live show and commercial brand activations. The example is Holy, a maker of drink powder that is mixed with water and shaken to make a drink.

The product-related action—shaking a drink shaker—becomes a shared audience choreography. The audience initiates the action, while the software coordinates how it advances and makes that progress visible as part of the show.

Setup: 3 × 3 Neighborhoods, each with 3 × 3 Agents

The stage or audience display shows a grid of nine Neighborhoods. Each Neighborhood contains nine Agents. The three flavor groups—**cherry**, **woodruff**, and **watermelon**—each occupy one column. The three Neighborhoods in a column form the bottle's three levels, from bottom to top.

SHOW DISPLAY			
	Cherry	Woodruff	Watermelon
Top	[NH]	[NH]	[NH]
Middle	[NH]	[NH]	[NH]
Bottom	[NH]	[NH]	[NH]
	↑	↑	↑
Fill level rises from bottom to top			

Each [NH] contains a 3 × 3 Agent grid. How Agents map to people, devices, or virtual participants depends on the specific show implementation.

Activation flow

1. **Start signal:** The bottom Neighborhoods are activated. Their Agents prompt the corresponding participants to shake their smartphones like shakers.
2. **Detect movement:** An app or connected detection service checks an agreed criterion, such as a minimum number of shakes or detected movement intensity within a time window.
3. **Threshold reached:** Once the threshold is met, the participant raises their smartphone overhead. This gesture confirms completion and causes the Agent to emit a Signal.
4. **Pass the Signal on:** The Signal activates the appropriately connected Agent in the Neighborhood above. That Agent begins the next part of the choreography and prompts the participants there to shake.
5. **Show the fill level:** Activation climbs upward level by level. Agents or Neighborhoods light up on the show display; from above, it looks as if a Holy bottle is filling.

6. **Finish:** When the top level is reached, the experience can trigger a shared animation, jingle, or call to action.

The three flavor columns can run independently or in sync. For example, the show could reveal which group fills its column first, or synchronize all groups for a shared finale.

What the Local Intelligence Layer contributes

- **Local reactions:** Each Agent responds to a local event, rather than requiring a central sequence to control every participant directly.
- **Signal propagation:** Confirmed progress can be passed selectively to the next level.
- **Configurable rules:** Shake thresholds, time windows, repetitions, and transition conditions can be configured as experiment rules.
- **Live visualization:** Aggregated Agent and Neighborhood states can drive the fill-level display and show group progress.
- **Reusability:** The same pattern can be adapted for other products, brands, stage visuals, and audience interactions.

Commercial use and measuring success

As a brand activation, the experience can turn a product message into a shared action. Possible aggregated metrics include:

- share of activated participants who complete the task
- time to reach each level and fill the entire bottle
- completion rate by flavor group
- number of Signals emitted and repeat participation
- response to different thresholds or show sequences

These metrics can help Crows and the brand partner compare choreography or activation variants. They should only be collected when measurement is transparently explained and necessary for the purpose.

Privacy, safety, and accessibility

- Participation should be voluntary, with a clear explanation before activation.

- If smartphone motion detection is used, the app should process only the movement data needed to evaluate the threshold. Storing raw data or linking it to an identifiable person is not necessary for an aggregated show visualization.
- Thresholds and time windows should be safe and adaptable to different abilities. An alternative gesture or manual confirmation can allow participation without shaking.
- The smartphone gesture should be performed with adequate space and without throwing or swinging the device dangerously.
- The show should continue even if some devices are offline or some participants choose not to take part.

Implementation boundary

The Local Intelligence Layer could orchestrate Agent and Neighborhood states, rules, and Signal propagation. Detecting smartphone motion, connecting to a show display, and rendering updates in real time require suitable app, hardware, or integration components. These interfaces and actual performance would need to be validated separately before a live deployment.